

ENCM 515: ASPA – Lab 3

Group #	23	Date: Mar 12, 2025
Student name(s)	Lee Evans	
	Sudarshan Naicker	
Instructor/TA		

To submit:

- Add your lab sheet (as PDF if possible) and any other added/modified .c or .h files to a zip archive and upload to D2L.

TASKS

Q1: Write the code for a function to implement the Biquad filter with the following coefficients: $b_0 = b_2 = 10158$, $b_1 = 20283$, $\{-a_1\} = -2261$, $\{-a_2\} = -10060$. Include an annotated version of your source code in your report. Take a screenshot of the assembly code for your function (after compiling), and explain which assembly instructions correspond to which parts of the filter's operation (e.g., what part is for managing the state variables (history of the input/output), what part corresponds to multiplication with the coefficients, etc.) How many cycles are required for processing a sample?

Audio without IIR filtering: "Now more music on your favorite station" with a significant amount of noise coupled.

Audio with IIR filtering: "Now more music on your favorite station" with a noticeable drop in noise.

```
ITM_Port32(0) = 1;
filteredSampleL = ProcessSample(newSampleL); // "L"
ITM_Port32(0) = 2;
```

SWV Trace Log:

Index	Type	Data	Cycles	Time(s)	Extra info
0	Sync	96	1999999	19.999990 ms	
1	ITM Port 30	0	2880735	28.807350 ms	
2	Sync	48	2881380 ?	?	No timestamp received ...
3	Sync	48	2881380 ?	?	No timestamp received ...
4	ITM Port 30	10	2881380	28.813800 ms	
5	ITM Port 0	1	2881525	28.815250 ms	
6	Sync	48	2881665 ?	?	No timestamp received ...
7	ITM Port 0	2	2881665	28.816650 ms	Timestamp delayed. Pa...

From the image above, we can see that it takes 1.40us and 140 cycles to process a single sample. Far faster than our sampling period that we discussed in lab2.

C code for IIR filter. No special MAC or SIMD functions were used for simplicity, readability and to allow space for optimization later in the lab.

```
//global variables to store IIR value history
int32_t previous_outputs[2] = {0,0};
int32_t previous_inputs[2] = {0,0};

//feedforward tap values
int32_t feedforward_taps[3] = {10158, 20283, 10158};
//feedback tap values
int32_t feedback_taps[2] = {-2261, -10060};

// processes data with IIR filter
static int16_t ProcessSample(int16_t newsample) {
    //where our output is stored
    int32_t accumulator = 0;

    //multiply and add feedforward values, b0, b1, and b2, respectively
    accumulator += (int32_t) newsample * feedforward_taps[0];
    accumulator += previous_inputs[0] * feedforward_taps[1];
    accumulator += previous_inputs[1] * feedforward_taps[2];

    //multiply and add feedback values, a-1 and a-2, respectively
    accumulator += previous_outputs[0] * feedback_taps[0];
    accumulator += previous_outputs[1] * feedback_taps[1];

    //shuffle input sample history
    previous_inputs[1] = previous_inputs[0];
    previous_inputs[0] = (int32_t) newsample;

    //shuffle output sample history
    previous_outputs[1] = previous_outputs[0];
    previous_outputs[0] = accumulator >> 16;

    //convert 32bit accumulator value to 16 bits and return
    return (int16_t) (accumulator >> 16);
}
```

In the image below we see the entry point for our IIR filter function after, the initialization of our accumulator variable after 0x08001350, then 5 steps of multiply and accumulate for each of our feedforward and feedback values each starting at 0x0800135e, 0x08001370, 0x08001382, 0x08001394 and 0x080013a6 respectively. We see that it takes 8 assembly operations for each multiply and add operation. Seeing this breakdown, its clear how a SIMD operation could speed things along.

```

393      static int16_t ProcessSample(int16_t newsample) {
      ProcessSample:
08001350:  push    {r7}
08001352:  sub     sp, #20
08001354:  add     r7, sp, #0
08001356:  mov     r3, r0
08001358:  strh    r3, [r7, #6]
395      int32_t accumulator = 0;
0800135a:  movs    r3, #0
0800135c:  str     r3, [r7, #12]
398      accumulator += (int32_t) newsample * feedforward_taps[0];
0800135e:  ldrrsh.w r3, [r7, #6]
08001362:  ldr     r2, [pc, #136] @ (0x80013ec <ProcessSample+156>)
08001364:  ldr     r2, [r2, #0]
08001366:  mul.w   r3, r2, r3
0800136a:  ldr     r2, [r7, #12]
0800136c:  add     r3, r2
0800136e:  str     r3, [r7, #12]
399      accumulator += previous_inputs[0] * feedforward_taps[1];
08001370:  ldr     r3, [pc, #124] @ (0x80013f0 <ProcessSample+160>)
08001372:  ldr     r3, [r3, #0]
08001374:  ldr     r2, [pc, #116] @ (0x80013ec <ProcessSample+156>)
08001376:  ldr     r2, [r2, #4]
08001378:  mul.w   r3, r2, r3
0800137c:  ldr     r2, [r7, #12]
0800137e:  add     r3, r2
08001380:  str     r3, [r7, #12]
400      accumulator += previous_inputs[1] * feedforward_taps[2];
08001382:  ldr     r3, [pc, #108] @ (0x80013f0 <ProcessSample+160>)
08001384:  ldr     r3, [r3, #4]
08001386:  ldr     r2, [pc, #100] @ (0x80013ec <ProcessSample+156>)
08001388:  ldr     r2, [r2, #8]
0800138a:  mul.w   r3, r2, r3
0800138e:  ldr     r2, [r7, #12]
08001390:  add     r3, r2
08001392:  str     r3, [r7, #12]
403      accumulator += previous_outputs[0] * feedback_taps[0];
08001394:  ldr     r3, [pc, #92] @ (0x80013f4 <ProcessSample+164>)
08001396:  ldr     r3, [r3, #0]
08001398:  ldr     r2, [pc, #92] @ (0x80013f8 <ProcessSample+168>)
0800139a:  ldr     r2, [r2, #0]
0800139c:  mul.w   r3, r2, r3
080013a0:  ldr     r2, [r7, #12]
080013a2:  add     r3, r2
080013a4:  str     r3, [r7, #12]
404      accumulator += previous_outputs[1] * feedback_taps[1];
080013a6:  ldr     r3, [pc, #76] @ (0x80013f4 <ProcessSample+164>)
080013a8:  ldr     r3, [r3, #4]
080013aa:  ldr     r2, [pc, #76] @ (0x80013f8 <ProcessSample+168>)
080013ac:  ldr     r2, [r2, #4]
080013ae:  mul.w   r3, r2, r3
080013b2:  ldr     r2, [r7, #12]
080013b4:  add     r3, r2
080013b6:  str     r3, [r7, #12]

```

In the image of assembly code below, we see the shuffling of our state variables for the previous inputs and outputs at 0x080013b8, 0x080013c0, 0x080013c8, and 0x080013d0 respectively. This only takes a few lines since our buffers are small, and finally we see the return of our new sample and the end of our function starting at 0x080013d8.

```

407      previous_inputs[1] = previous_inputs[0];
080013b8:  ldr    r3, [pc, #52]    @ (0x80013f0 <ProcessSample+160>)
080013ba:  ldr    r3, [r3, #0]
080013bc:  ldr    r2, [pc, #48]    @ (0x80013f0 <ProcessSample+160>)
080013be:  str    r3, [r2, #4]
408      previous_inputs[0] = (int32_t) newsample;
080013c0:  ldrsh.w r3, [r7, #6]
080013c4:  ldr    r2, [pc, #40]    @ (0x80013f0 <ProcessSample+160>)
080013c6:  str    r3, [r2, #0]
411      previous_outputs[1] = previous_outputs[0];
080013c8:  ldr    r3, [pc, #40]    @ (0x80013f4 <ProcessSample+164>)
080013ca:  ldr    r3, [r3, #0]
080013cc:  ldr    r2, [pc, #36]    @ (0x80013f4 <ProcessSample+164>)
080013ce:  str    r3, [r2, #4]
412      previous_outputs[0] = accumulator>>16;
080013d0:  ldr    r3, [r7, #12]
080013d2:  asrs   r3, r3, #16
080013d4:  ldr    r2, [pc, #28]    @ (0x80013f4 <ProcessSample+164>)
080013d6:  str    r3, [r2, #0]
415      return (int16_t) (accumulator >> 16);
080013d8:  ldr    r3, [r7, #12]
080013da:  asrs   r3, r3, #16
080013dc:  sxth   r3, r3
416  }

```

Q2> Analyze the code that you have written and use one or more techniques to optimize different parts of the code. Explain which technique(s) you've used and why you've used them. How many cycles per sample do you achieve after using your optimizations?

The first optimization, seen in the code below, was to use a MAC function that combines the multiply and accumulate operations. I expect this to shave off a few cycles, but not many since we only perform 5 MAC operations. The code for this is seen below.

```

// processes data with IIR filter
static int16_t ProcessSample(int16_t newsample) {
    //where our output is stored
    int32_t accumulator = 0;

    /* Using MAC function for feedforward*/
    __asm volatile ("SMLABB %[result], %[op1], %[op2], %[acc]"
: [result] "+r" (accumulator)
: [op1] "r" ((int32_t) newsample), [op2] "r" (feedforward_taps[0]), [acc] "r" (accumulator)
);

    __asm volatile ("SMLABB %[result], %[op1], %[op2], %[acc]"
: [result] "+r" (accumulator)
: [op1] "r" (previous_inputs[0]), [op2] "r" (feedforward_taps[1]), [acc] "r" (accumulator)
);

    __asm volatile ("SMLABB %[result], %[op1], %[op2], %[acc]"
: [result] "+r" (accumulator)
: [op1] "r" (previous_inputs[1]), [op2] "r" (feedforward_taps[2]), [acc] "r" (accumulator)
);

    /* Using MAC function for feedback*/
    __asm volatile ("SMLABB %[result], %[op1], %[op2], %[acc]"
: [result] "+r" (accumulator)
: [op1] "r" (previous_outputs[0]), [op2] "r" (feedback_taps[0]), [acc] "r" (accumulator)
);

    __asm volatile ("SMLABB %[result], %[op1], %[op2], %[acc]"
: [result] "+r" (accumulator)
: [op1] "r" (previous_outputs[1]), [op2] "r" (feedback_taps[1]), [acc] "r" (accumulator)
);

    //shuffle input sample history
    previous_inputs[1] = previous_inputs[0];
    previous_inputs[0] = (int32_t) newsample;

    //shuffle output sample history
    previous_outputs[1] = previous_outputs[0];
    previous_outputs[0] = accumulator>>16;

    //convert 32bit accumulator value to 16 bits and return
    return (int16_t) (accumulator >> 16);
}

```

Index	Type	Data	Cycles	Time(s)	Extra info
0	Sync	56	1999999	19.999990 ms	
1	ITM Port 30	0	2880118	28.801180 ms	
2	Sync	48	2880762 ?	?	No timestamp received ...
3	Sync	48	2880762 ?	?	No timestamp received ...
4	ITM Port 30	10	2880762	28.807620 ms	
5	ITM Port 0	1	2880907	28.809070 ms	
6	Sync	48	2881038 ?	?	No timestamp received ...
7	ITM Port 0	2	2881038	28.810380 ms	Timestamp delayed. Pa...

From the image above, we can see that it takes 1.31us and 131 cycles to process a single sample now. Not a significant improvement, but an improvement nonetheless.

I attempted to further optimize with a SIMD SMLAD operation to perform 2 feedback and feedforward MAC operations simultaneously, but my attempts resulted in slightly more noise remaining on the output than before, but still less than the original audio. I believe this is due to a data type issue causing. If I have additional time at the end of the lab I will come back to this problem, but for now the question is answered.

Q3> Implement a new function that produces an output with echo. Aim for a delay of around 125 ms. How large does your history (state variable array) have to be for this delay? Is this an FIR or IIR filter? Add to your report an analysis of the compiled code. Discuss any “optimizations” that you needed to use to make this function work under the time constraints.

Since the sample period is 125us, to delay 125ms we need an array of size 125ms/125us, or 1000 samples.

$$y[n] = x[n] + \alpha x[n - n_0]$$

Since this function only uses feedforward, it is considered a FIR filter.

I first attempted implementing the equation above with a basic buffer shuffling operation in a for loop, running through it 1000 times to move my data around. This failed to meet timing requirements, so the output became garbled. I needed a method to speed up or optimize array navigation. I next tested a circular buffer to speed up buffer navigation. Instead of moving around 1000 variables, I now only needed to change 1 buffer element and recalculate an index for buffer movement every sample processed.

Below is this implementation. It has a similar structure to the previous question where theres a buffer that is pulled from and managed, some manipulation of our data, a saturation check, and returning the value. Since the IIR lowpass filter has such little impact to the total cycle count, I chose to preprocess my data before implementing the echo effect.

```

int16_t echo_buffer[1000];
uint16_t echo_buff_len = 1000;
uint16_t echo_index = 0;
float gain = 0.7;

static int16_t echoEffect(int16_t newsample) {
    //this adds ~131 cycles or 131us of time
    int16_t filtered_sample = ProcessSample(newsample);

    //add current sample to sample from 125ms ago. This creates the echo.
    int32_t result = filtered_sample + 0.8*echo_buffer[echo_index];

    //store the new sample in the buffer overwriting the oldest sample
    echo_buffer[echo_index] = filtered_sample;

    //update the index of the circular buffer
    echo_index = (echo_index + 1) % echo_buff_len;

    //Check and correct if we have overflow
    if(result > 32767){
        result = 32767;
    } else if (result < -32768){
        result = -32768;
    }

    return (int16_t) result;
}

```

Index	Type	Data	Cycles	Time(s)	Extra info
0	Sync	56	1999999	19.999990 ms	
1	ITM Port 30	0	2875940	28.759400 ms	
2	ITM Port 0	1	2876266	28.762660 ms	
3	Sync	48	2876589 ?	?	No timestamp received for packet, cycles value guessed.
4	ITM Port 0	2	2876589	28.765890 ms	Timestamp delayed. Packet delayed.

Looking at timing we can see that the function takes 323 cycles and 3.23us.

Curious about how assembly instructions might help speed things up, but lacking much knowledge of assembly operation I searched google and prompted ChatGPT for some ideas. After some poking around, I found block transfer style instructions. Below is a code snippet of the code formatted with a bit of help from Chat.

```

int16_t echo_buffer[1000];
uint16_t echo_buff_len = 1000;
float gain = 0.7;
static int16_t echoEffect(int16_t newsample) {
    //this adds ~131 cycles or 131us of time
    int16_t filtered_sample = ProcessSample(newsample);

    //set the head of our buffer with the new sample
    echo_buffer[0] = filtered_sample;

    // Add current sample to sample from 125ms ago. This creates the echo.
    int32_t result = filtered_sample + 0.8*echo_buffer[echo_buff_len - 1];

    // Shift our buffer along for the next sample
    __asm volatile (
        "mov     r2, %[echo_buff_len]    \n\t" // r2 = len
        "subs    r2, r2, #1              \n\t" // r2 = len - 1 (number of moves)
        "lsls    r2, r2, #1              \n\t" // r2 = (len - 1) * 2 (convert count to bytes; ea
        "add     r1, %[echo_buffer], r2  \n\t" // r1 = address of last element to be moved (in b
        "1:                                             \n\t"
        "subs    r1, r1, #2              \n\t" // decrement r1 by 2 bytes (move to previous elem
        "ldrsh   r3, [r1]                \n\t" // load the halfword (16-bit value) from the curr
        "strh    r3, [r1, #2]            \n\t" // store the halfword into the destination address
        "cmp     r1, %[echo_buffer]      \n\t" // compare current source pointer with the base o
        "bne     1b                      \n\t" // if not equal, repeat the loop
        : [echo_buffer] "r" (echo_buffer), [echo_buff_len] "r" (echo_buff_len)
        : "r1", "r2", "r3", "memory"
    );

    //Check and correct if we have overflow
    if(result > 32767){
        result = 32767;
    } else if (result < -32768){
        result = -32768;
    }

    return (int16_t) result;
}

```

Index	Type	Data	Cycles	Time(s)
0	Sync	56	1999999	19.999990 ms
1	ITM Port 30	0	2878337	28.783370 ms
2	ITM Port 0	1	2878661	28.786610 ms
3	ITM Port 0	2	2885967	28.859670 ms
4	Sync	48	0	0.000000 s

Looking at the timing report above, we see that it took 7306 cycles and 73.06us. Far slower than the circular buffer, but it did still pass timing requirements. Not useful for this application but still interesting.

Q4> Implement a new function that produces an output with reverb. Is this an FIR or IIR filter? Add to your report an analysis of the compiled code. Add to your report an analysis of the compiled code. Discuss any “optimizations” that you needed to use to make this function work under the time constraints.

Since this function uses feedback, it would be considered a IIR filter.

$$y[n] = x[n] + \alpha y[n - n_0]$$

The reverb equation above has the same format as the previous echo filter, except we are using the past outputs to influence the future outputs. Therefore we can reuse the same code.

After changing some variable names and building the buffer from the past outputs, we have the code below.

Unsurprisingly this worked the first try since we worked out the kinks and got the timing right for our echo function.

To further optimize, I implemented QADD16 to handle addition of my old output and new output. The benefit is that QADD16 handles saturation arithmetic better than my two 'if' statements. Potentially shaving off a few more cycles!

```
int16_t reverb_buffer[1000];
uint16_t reverb_buff_len = 1000;
uint16_t reverb_index = 0;

static int16_t reverbEffect(int16_t newsample) {
    //this adds ~131 cycles or 131us of time
    int16_t filtered_sample = ProcessSample(newsample);

    int16_t delayed_sample = 0.7*reverb_buffer[reverb_index];

    int16_t result;

    //add current sample to output from 125ms ago. This creates the reverb.
    //QADD16 performs addition of 16bit ints and handles saturation arithmetic
    __asm volatile (
        "QADD16 %[result], %[filtered_sample], %[delayed_sample]    \n\t"
        : [result] "=r" (result)
        : [filtered_sample] "r" (filtered_sample), [delayed_sample] "r" (delayed_sample)
    );

    //store the new sample in the buffer overwriting the oldest sample
    reverb_buffer[reverb_index] = result;

    //update the index of the circular buffer
    reverb_index = (reverb_index + 1) % reverb_buff_len;

    return result;
}
```

Index	Type	Data	Cycles	Time(s)
0	Sync	56	1999999	19.999990 ms
1	Sync	48	2877709 ?	?
2	ITM Port 30	0	2877709	28.777090 ms
3	ITM Port 0	1	2878033	28.780330 ms
4	Sync	48	2878232 ?	?
5	ITM Port 0	2	2878232	28.782320 ms

Looking at the timing report above, we see that it took 199 cycles and 1.99us. This filter appears to benefit greatly through optimizing saturation arithmetic with QADD16, considering essentially the same code in the echo filter without QADD16 required 323 cycles.

Q5> Write a reflection of what you have observed and learned in this lab, drawing connections between what we've discussed in class and what you've done in this lab.

I found it really interesting to compare the low pass FIR filter we implemented in lab2 with the IIR lowpass filter we implemented in this lab. Especially the reduction in processing time with the reduced amount of taps used. Some things I'm left interested in exploring is testing the previous FIR lowpass filter on our audio sample to see what the performance is compared to our IIR filter. Something else that I'm curious about and might explore later is how to generate the tap values for our FIR and IIR filters, as well as seeing under what conditions our IIR filter would become unstable, but this is beyond the scope of this lab.

I found the implementation of a circular buffer very neat. It made me think about how this type of buffer wouldn't have worked for our FIR filter since position of array elements was essential, but was ideal for our echo and reverb effect since we only wanted to pull a single value from the buffer at a time.

I'm continuously impressed by how ARM assembly instructions can be used to speed up even single lines of code to have a meaningful impact on cycle count. I spent a good amount of time browsing through the available assembly operations found in the link below and while I have a hard time understanding what most of the operations are used for, I think having a vague background on them will allow me to come back to this resource to find solutions to other potential bottle necks in my code!

<https://developer.arm.com/documentation/ddi0406/cb/Application-Level-Architecture/Instruction-Details/Alphabetical-list-of-instructions>

My favourite part of this lab was the ability to write code and immediately validate its function through comparing our audio output on the left and right channels audio output. It would be cool to streamline a process that also allows me to analyze the filtered output through a spectral analysis, but exporting data and importing into a Python file is pretty time consuming. Perhaps connecting the audio output to a oscilloscope that can perform an FFT would be another cool step to see how noise is being reduced through our digital filtering!

Overall I found this to be a great lab to help me develop skills and understanding that I can apply outside of this course!